Discovery Executive Summary

Project: test · **Generated:** 7/20/2026, 6:41:12 PM

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Backend Modernization Analysis	MODERATE	—

---

1. Backend Modernization Analysis

OVERALL CODEBASE RATING — BACKEND MODERNIZATION

Moderate

The verdict is driven by inline persistence/business logic in route handlers, the in-memory job store, and missing API governance across exposed services.

EXECUTIVE SUMMARY

The backend stack is a Node/Express ecosystem split across a client gateway, a client integrations API, a vendor orchestration API, and a vendor license service. The most important modernization gap is architectural rather than syntactic: request handlers and route modules still carry orchestration, persistence, and auth concerns inline, while the orchestration service relies on mutable in-memory job state for production workflow coordination. API governance is present in the sense that the system exposes multiple HTTP surfaces, but there is no observed OpenAPI/spec linting or contract-test discipline in the scanned backend services. Overall health is Moderate: the code is organized into services, but the persistence boundary and lifecycle boundary are still leaky enough to create operational risk and make future changes harder than they should be.

4.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating

H1	Dynamic Variable Creation	Dynamic-var-from-input occurrences	0	1–10	>10	0 observed	GOOD
H2	Global Mutable State	Globals / mutable static state	0	1–5	>5	1 mutable in-memory job store	MODERATE
H3	Direct SQL Outside Data Layer	Data-layer compliance %	>90%	60–90%	<60%	4 route files issue ORM calls inline	MODERATE
H4	Static / Singleton Abuse	Business-logic static/singleton classes	0	1–5	>5	0 observed	GOOD
H5	Missing Service Layer	Handlers with inline business logic	<10	10–20	>20	6 handlers/routes inline	MODERATE
H6	API Sprawl	Documented & governed endpoints %	>90%	80–90%	<80%	N/A — no duplicate contract evidence observed	GOOD
H7	Missing API Governance	Governance compliance %	100%	90–99%	<90%	N/A — no OpenAPI/contract-test evidence observed	HIGH RISK

4.5 Actions Required

Hotspot	Action	Rating	Priority
H2 Global Mutable State	Replace the in-memory `jobStore` map with an injected repository backed by durable storage and atomic token consumption.	MODERATE	MEDIUM
H3 Direct SQL / ORM Outside Data Layer	Move `Subscription` and `Connector` lookups into repository modules, then have routes call services instead of querying models directly.	MODERATE	MEDIUM
H5 Missing Service Layer	Extract job-preparation, entitlement, and connector bootstrap workflows into services and keep route handlers thin.	MODERATE	MEDIUM
H7 Missing API Governance	Publish OpenAPI specs for the exposed HTTP surfaces and enforce them with linting and contract tests in CI.	HIGH RISK	CRITICAL

4.6 Expected Outcomes

- Request handlers become thinner and easier to test because validation, workflow logic, and persistence move into dedicated layers.
- The job lifecycle becomes safer under restart and scale-out because token state is no longer held in process memory.
- Reusing repositories and services across routes reduces duplication between the gateway, orchestration API, and license service.

- OpenAPI and contract tests make breaking API changes visible before they ship, which lowers integration risk for the gateway and client consumers.